AI Reference Report — Web Mario (Cocos Creator)

1. AI Tool(s) Used

- **Claude (Anthropic)** — claude-sonnet-4-6
- Used for code generation, debugging, and helping with Firebase integration.

---

2. Scope of Usage

Usage 1 — Firebase Hosting Deployment

Location: `firebase.json`

Prompt:

> "I built a Cocos Creator web game. The build output is in `build/web-mobile`. How do I deploy it to Firebase Hosting? Give me the firebase.json config and the CLI commands."

AI Response (summarized): The AI generated the `firebase.json` configuration pointing `"public"` to `"build/web-mobile"`, added a `"headers"` block to disable JS caching, and provided the terminal commands:

```
firebase login
firebase init hosting
firebase deploy
```

Refined Version (`firebase.json`):

```
{
  "hosting": {
    "public": "build/web-mobile",
    "ignore": ["firebase.json", "**/.*", "**/node_modules/**"],
    "headers": [
      {
        "source": "**/*.js",
        "headers": [{"key": "Cache-Control", "value": "no-cache"}]
      }
    ]
  }
}
```

Refinement & Explanation: The AI's initial config didn't include the `no-cache` header. I added it manually because Cocos Creator generates hashed filenames (e.g., `main.d3bfb.js`) but `index.html` itself has no hash — without `no-cache`, browsers would cache the old HTML and load stale JS bundles after a redeploy. Adding this rule forces browsers to revalidate JS files on every load so players always get the latest version.

Usage 2 — Firestore Leaderboard (Submit & Fetch Scores)

Location: `assets/Script/FirebaseService.ts`

Prompt:

> "Write a TypeScript service class for Cocos Creator that submits a player score to Firestore (REST API, no SDK) and retrieves the top 10 scores sorted by score descending. Use XMLHttpRequest only. The Firestore project ID is mario-509a8, collection is 'leaderboard'."

AI Response (summarized): The AI generated `submitScore()` (POST to Firestore REST with `stringValue`/`integerValue` fields) and `getLeaderboard()` (GET with `pageSize=100`, map documents to `{name, score}`, sort descending, slice top 10). A private `_fetch()` helper handled XHR for both methods.

Refined Version (`getLeaderboard` excerpt):

```
static getLeaderboard(): Promise<{name: string, score: number}[]> {
    const url = `${BASE_URL}/leaderboard?pageSize=100`;
    return new Promise((resolve, reject) => {
        FirebaseService._fetch(url, "GET", null, (data: any) => {
            const docs = data.documents || [];
            const results = docs.map((doc: any) => ({
                name:  doc.fields?.name?.stringValue  || "???",
                score: parseInt(doc.fields?.score?.integerValue || "0")
            }));
            results.sort((a: any, b: any) => b.score - a.score);
            resolve(results.slice(0, 10));
        }, reject);
    });
}
```

Refinement & Explanation: The AI originally used `Number()` to parse the score. I changed it to `parseInt()` because Firestore stores integers as strings (`integerValue: "1200"`), and `parseInt` is more correct for integer parsing. I also added the `|| "0"` fallback manually to prevent `NaN` showing up in the leaderboard UI when a document is missing the score field.

Usage 3 — Player Ground Detection (Raycasting)

Location: `assets/Script/PlayerController.ts` (`checkGround` method)

Prompt:

> "In Cocos Creator with Box2D, how do I properly detect if the player is standing on the ground? Using onEndContact is unreliable and sometimes Mario floats. Is there a better way?"

AI Response (summarized): The AI suggested using `PhysicsManager.rayCast()` instead of relying on contact callbacks. It cast a short ray downward from Mario's feet each frame and checked if it hit anything. If the ray hits a collider (that isn't Mario himself) the player is considered on the ground.

Refined Version (`checkGround` method):

```
private checkGround() {
    const worldPos = this.node.convertToWorldSpaceAR(cc.v2(0, 0));
    const pm = cc.director.getPhysicsManager();
    const footY  = worldPos.y - 8;
    const checkY = worldPos.y - 12;
    const hit = (ox: number) => {
        const r = pm.rayCast(cc.v2(worldPos.x + ox, footY), cc.v2(worldPos.x +
ox, checkY), cc.RayCastType.Closest);
        return r && r.length > 0 && r[0].collider.tag !== 2 && r[0].collider.tag
!== 5 && r[0].collider.tag !== 10;
    };
    const center = hit(0);
    const left   = hit(-3);
    const right  = hit(3);
    this.onGround = center || (left && right) || left || right;
}
```

Refinement & Explanation: The AI's initial version only cast one ray from the center of Mario's feet. This caused a bug where Mario would fall off platform edges even though his foot was still touching — because the single center ray missed the edge. I changed it to cast three rays (center, left offset, right offset) so Mario stays grounded as long as any part of his foot touches a surface. I also added tag filtering to exclude colliders tagged as enemies (tag 2), coins (tag 5), and shells (tag 10) from the ground check, which stopped Mario from getting stuck on top of enemies.

Usage 4 — GameData Singleton (Persistent Data Across Scenes)

Location: `assets/Script/GameData.ts`

Prompt:

> "In Cocos Creator, how do I keep score, lives, and coins data when changing scenes? The data resets every time the scene changes."

AI Response (summarized): The AI explained `cc.game.addPersistRootNode()` — a Cocos Creator API that keeps a node alive across scene transitions. It suggested putting a singleton component on that node to store all persistent game state. The pattern uses a static `inst` getter so any script can access the data without a node reference.

Refined Version (`GameData.ts`):

```
@ccclass
export default class GameData extends cc.Component {
    private static _inst: GameData = null;
    static get inst() { return GameData._inst; }

    score: number = 0;
    lives: number = 3;
    coins: number = 0;
    level: number = 1;

    onLoad() {
        if (GameData._inst) { this.node.destroy(); return; }
        GameData._inst = this;
        cc.game.addPersistRootNode(this.node);
    }

    reset() {
```

```
        this.score = 0;
        this.lives = 3;
        this.coins = 0;
        this.level = 1;
    }
}
```

Refinement & Explanation: The AI's first version didn't handle the case where the GameData node already existed (e.g., when going back to the main menu and reloading a scene). Without the guard `if (GameData._inst) { this.node.destroy(); return; }`, a second GameData node would be created and both would survive — causing the score and lives to appear duplicated or reset unexpectedly. Adding the early-destroy check ensures there is always exactly one instance.

Usage 5 — Koopa Shell State Machine

Location: `assets/Script/Enemy_Koopa.ts`

Prompt:

> "Write a Cocos Creator TypeScript script for a Koopa Troopa enemy. It should walk, turn into an idle shell when stomped, slide as a fast shell when kicked, bounce off walls, kill other Goombas it hits while sliding, and destroy itself if it falls off the edge. Use a state machine with states: walk, shell_idle, shell_move."

AI Response (summarized): The AI produced a three-state machine (`walk | shell_idle | shell_move`) using a `KoopaState` type alias. It included `onBeginContact` collision logic for distinguishing stomp-from-above vs. side-hit, a `flip()` method with a cooldown to prevent instant double-flip, edge detection via raycasting, and a `killByShell()` method so other enemies could be killed by a sliding shell.

Refined Version (`kickShell` excerpt):

```
private kickShell(marioNode: cc.Node) {
    const marioX = marioNode.convertToWorldSpaceAR(cc.v2(0, 0)).x;
    const myX    = this.node.convertToWorldSpaceAR(cc.v2(0, 0)).x;
    this.shellSpeed = marioX < myX ? 200 : -200;
    this.state = "shell_move";
    if (this.anim) this.anim.play("koopa_shell");
    const col = this.getComponent(cc.PhysicsBoxCollider);
    if (col) col.tag = 10;
    const pc = marioNode.getComponent("PlayerController") as any;
    if (pc) pc.rb.linearVelocity = cc.v2(pc.rb.linearVelocity.x, 300);
}
```

Refinement & Explanation: The AI's initial version kicked the shell in a hardcoded direction (always right). I fixed it to compare Mario's world X position against the shell's world X position so the shell always flies away from Mario regardless of which side he kicked from. The collider `tag` was also missing from the AI output — without setting it to `10` on kick, `PlayerController` would treat the moving shell as a walking Koopa and apply wrong damage logic. I also added a small upward bounce (`linearVelocity.y = 300`) to give better feel when kicking.

Report prepared: May 2026